

## GHID DE ÎNVĂȚARE PENTRU NOTEBOOKLM

OffByOne Academy - pregătire pentru prezentare și întrebări

Scop:

Acest document este făcut pentru a fi încărcat în NotebookLM împreună cu prezentarea scurtă și documentul cu întrebări. Rolul lui este să îți transforme proiectul într-un material ușor de ascultat ca audio, podcast sau recapitulare video. Nu trebuie memorat cuvânt cu cuvânt. Trebuie înțeles firul logic.

Fișiere recomandate de încărcat în NotebookLM:

1. Prezentare\_OffByOne\_Scurta.pdf
2. Intrebari\_Simpozion\_OffByOne.pdf
3. Ghid\_NotebookLM\_OffByOne\_Academy.txt
4. Opțional: fișierele relevante din proiect\_documentatie, mai ales materialele despre metode de sortare, algoritmi fundamentali și tehnici algoritmice.

Prompt recomandat pentru NotebookLM:

"Folosind aceste surse, fă-mi un podcast în limba română, de 15-20 de minute, pentru pregătirea prezentării proiectului OffByOne Academy. Explică-mi ideea proiectului, arhitectura, securitatea, partea de AI, vizualizatorul, partea pedagogică și întrebările grele pe care le-ar putea pune comisia. Pune accent pe metodele de sortare și pe diferențele dintre Bubble Sort, Selection Sort, Insertion Sort, Quick Sort, Merge Sort și Counting Sort. După podcast, generează-mi o simulare de interviu cu 20 de întrebări și răspunsuri scurte."

### 1. IDEEA PROIECTULUI ÎNTR-O FRAZĂ

OffByOne Academy este o platformă web educațională pentru învățarea algoritmilor, cu accent pe metode de sortare, care combină teorie, cod C++, vizualizare interactivă, grile, parcursuri de învățare, profil cu progres, gamification și asistență AI.

Varianta scurtă pentru prezentare:

"Am construit o platformă care transformă algoritmii de sortare din concepte abstracte în experiențe interactive: vezi pașii pe Canvas, urmărești pseudocodul, rezolvi grile, primești feedback și ai un Profesor AI care răspunde pe baza documentației proiectului."

Ideea principală:

Problema nu este că elevii nu au acces la teorie. Problema este că teoria, codul, animația, testarea și feedback-ul sunt de obicei separate. OffByOne Academy le pune în aceeași aplicație.

## 2. POVESTEA PREZENTĂRII, PE SLIDE-URI

Slide 1 - Cine suntem și ce prezentăm

Proiectul se numește OffByOne Academy. Este o platformă web interactivă pentru studiul algoritmilor de sortare.

Cum spui simplu:

"Proiectul pornește de la o problemă didactică: algoritmii sunt greu de înțeles doar din pseudocod static. Am creat o platformă în care utilizatorul vede algoritmul în acțiune și poate exersa imediat."

Slide 2 - Problema și soluția

Probleme:

- algoritmii sunt abstracți;
- feedback-ul vine târziu;
- motivația scade repede;
- elevul trebuie să sară între multe site-uri: teorie, cod, compilator, grile.

Soluția OffByOne:

- teorie și cod C++;
- vizualizator interactiv;
- grile cu evaluare automată;
- Profesor AI, quiz AI și feedback pe cod;
- profil, badge-uri și streak-uri;
- PWA pentru acces mai ușor.

Răspuns bun dacă ești întrebat "care e noutatea?":

"Noutatea nu este un singur element izolat, ci integrarea lor. Aceeași platformă oferă teorie, execuție vizuală, testare, progres și AI contextualizat pe documentația proiectului."

### Slide 3 - Tehnologii

#### Backend:

- PHP 8.1+;
- MySQL 8.0;
- Apache;
- Docker Compose;
- prepared statements cu MySQLi.

#### Frontend:

- HTML5 Canvas;
- JavaScript vanilla;
- CSS cu design tokens;
- PWA cu manifest și service worker.

#### AI:

- Groq Cloud;
- Llama 3.3 70B;
- trei module: chat, quiz și code feedback.

#### Securitate:

- parole hash-uite cu password\_hash;
- CSRF token;
- CSP cu nonce;
- rate limiting;
- sesiuni cu timeout;

- audit și fix-uri succesive.

Referințe concrete din proiect:

- site\_g/index.php: front-controller, CSP, nonce, rute și layout comun.
- site\_g/PHP/helpers.php: CSRF, sesiune, mail, helper-e de securitate.
- site\_g/PHP/profesor\_ai\_chat.php: chat AI cu context din documentație.
- site\_g/PHP/ai\_quiz\_api.php: quiz AI, login obligatoriu, salvare scor.
- site\_g/PHP/documentation\_context.php: căutare în indexul proiect\_documentatie.
- site\_g/JS/visualizer.js: vizualizatorul Canvas.
- site\_g/pagini/profil.php: profil, progres, acuratețe, streak, istoric quiz AI.
- site\_g/manifest.json și site\_g/sw.js: PWA.

Slide 4 - Vizualizatorul interactiv

Vizualizatorul este "inima" aplicației.

Cum funcționează:

1. Pagina setează algoritmul prin data-algorithm.
2. Clasa SortingVisualizer citește algoritmul ales.
3. Generează un vector de valori.
4. Execută algoritmul instrumentat pas cu pas.
5. La fiecare pas actualizează barele din Canvas.
6. Actualizează contorii: comparații, schimbări, timp și stare.
7. Evidențiază linia de pseudocod activă.
8. Poate reda sunete cu Web Audio API.

Răspuns scurt:

"Nu este doar o animație preînregistrată. Utilizatorul poate schimba numărul de elemente, viteza, datele și algoritmul, iar vizualizatorul recalculează pașii în timp real."

## Slide 5 - Inteligență artificială

Există trei integrări AI:

1. Profesor AI: răspunde în română la întrebări despre algoritmi.
2. Quiz Generator: generează întrebări pe baza documentației.
3. Code Feedback: analizează fragmente de cod C++.

Ce este important:

AI-ul nu răspunde complet la întâmplare. În profesor\_ai\_chat.php se construiește un context extras din proiect\_documentatie prin documentation\_context.php. Asta înseamnă că răspunsurile sunt ancorate în fișierele proiectului.

La quiz AI, cerințele sunt stricte:

- întrebările trebuie să fie în română;
- trebuie să se bazeze pe documentația proiectului;
- trebuie să evite întrebări de memorare de tip "ce metodă este folosită în funcția X";
- trebuie să includă fragmente de cod când întreabă despre variabile sau condiții;
- testul AI necesită autentificare;
- scorul se salvează și apare în profil.

Răspuns scurt dacă ești întrebat de halucinații:

"Am redus riscul prin context local din proiect\_documentatie, prompt restrictiv, validarea structurii JSON pentru quiz, fallback-uri controlate și prin faptul că AI-ul este tratat ca asistent didactic, nu ca sursă absolută."

## Slide 6 - Diferențiatori

Față de platforme existente:

- VisuAlgo are vizualizări bune, dar nu oferă același pachet integrat în română cu AI contextualizat.
- GeeksForGeeks și W3Schools sunt bune ca documentație, dar sunt generale și în engleză.
- PBInfo este excelent pentru probleme și teorie, dar OffByOne pune accent pe vizualizare, progres, AI și experiență ghidată.

Ce spui comisiei:

"Nu am încercat să înlocuim PBIInfo sau VisuAlgo. Am combinat puncte utile într-o platformă orientată pe învățare ghidată, în română, cu feedback rapid."

#### Slide 7 - Rezultate și direcții viitoare

Rezultate:

- aproximativ 14K linii de cod;
- 13 tabele MySQL;
- 10 badge-uri;
- 6 algoritmi de sortare și 4 tehnici algoritmice;
- sistem de progres, streak, profil și istoric AI quiz.

Direcții viitoare:

- grafuri: BFS, DFS, Dijkstra;
- arbori: BST, AVL;
- internaționalizare română/engleză;
- monitorizare cu Sentry;
- backup automat al bazei de date;
- studiu pedagogic A/B.

#### Slide 8 - Final

Finalul trebuie să fie scurt:

"Mulțumim. În continuare putem arăta o demonstrație practică: alegem un algoritm, îl rulăm în laborator, apoi testăm Profesorul AI sau un quiz."

### 3. CE TREBUIE SĂ ȘTII FOARTE BINE DESPRE SORTĂRI

Proiectul este despre algoritmi de sortare, iar în categoria voastră există și o lucrare despre "Studiu comparativ al algoritmilor de sortare". Este probabil să primești întrebări pe această zonă.

### 3.1 Noțiuni generale

Sortarea înseamnă rearanjarea elementelor după o cheie, de obicei crescător sau descrescător.

Noțiuni utile:

- comparație: verificarea relației dintre două elemente;
- interschimbare / swap: schimbarea poziției a două elemente;
- stabilitate: elementele egale își păstrează ordinea relativă;
- in-place: algoritmul sortează fără memorie auxiliară proporțională cu  $n$ ;
- naturalețe: algoritmul se comportă mai bine când vectorul este aproape sortat;
- complexitate: cât crește timpul sau memoria în funcție de  $n$ .

### 3.2 Bubble Sort

Idee:

Compară elemente vecine și le interschimbă dacă sunt în ordine greșită. După fiecare trecere, un element mare ajunge spre final.

Complexitate:

- caz bun:  $O(n)$ , dacă există oprire când nu mai apar schimbări;
- caz mediu/rau:  $O(n^2)$ ;
- memorie:  $O(1)$ ;
- stabil: da, dacă interschimbi doar când  $v[j] > v[j+1]$ .

Întrebare probabilă:

"De ce Bubble Sort nu e potrivit pentru date mari?"

Răspuns:

"Pentru că face multe comparații repetate între vecini și ajunge la  $O(n^2)$ . La  $n$  mare, numărul de pași crește foarte repede."

### 3.3 Selection Sort

Idee:

La fiecare pas caută minimul din zona nesortată și îl pune pe poziția curentă.

Complexitate:

- toate cazurile:  $O(n^2)$ ;
- memorie:  $O(1)$ ;
- stabil: de obicei nu, din cauza swap-ului.

Întrebare probabilă:

"De ce Selection Sort face tot  $O(n^2)$  chiar dacă vectorul este deja sortat?"

Răspuns:

"Pentru că el caută minimul în zona rămasă indiferent de ordinea inițială. Nu poate ști fără să compare că minimul este deja pe poziția corectă."

### 3.4 Insertion Sort

Idee:

Construiește treptat o zonă sortată. Ia un element și îl inserează la locul potrivit în stânga.

Complexitate:

- caz bun:  $O(n)$ , dacă vectorul este deja sau aproape sortat;
- caz rău:  $O(n^2)$ , dacă vectorul este invers sortat;
- memorie:  $O(1)$ ;
- stabil: da.

Întrebare probabilă:



"De ce Insertion Sort poate fi bun pe vectori mici?"

Răspuns:

"Pentru că are overhead mic și se comportă foarte bine pe date aproape sortate. Mulți algoritmi practici îl folosesc pentru subvectori mici."

### 3.5 Quick Sort

Idee:

Alege un pivot, partiționează vectorul în valori mai mici și mai mari decât pivotul, apoi sortează recursiv zonele.

Complexitate:

- caz mediu:  $O(n \log n)$ ;
- caz rău:  $O(n^2)$ , dacă pivotul împarte foarte dezechilibrat;
- memorie:  $O(\log n)$  în medie pentru stiva recursivă;
- stabil: nu în varianta standard.

Întrebare probabilă:

"Când ajunge Quick Sort la  $O(n^2)$ ?"

Răspuns:

"Când pivotul este ales prost repetat și împarte vectorul în părți foarte dezechilibrate, de exemplu 0 și  $n-1$  elemente."

### 3.6 Merge Sort

Idee:

Împarte vectorul în jumătăți, sortează recursiv jumătățile, apoi le interclasează.

Complexitate:

- toate cazurile:  $O(n \log n)$ ;
- memorie:  $O(n)$ , pentru vector auxiliar;

- stabil: da, dacă la egalitate alegi elementul din stânga.

Întrebare probabilă:

"De ce Merge Sort are nevoie de memorie auxiliară?"

Răspuns:

"Pentru că interclasarea a două secvențe sortate se face de obicei într-un vector auxiliar, apoi rezultatul este copiat înapoi."

### 3.7 Counting Sort

Idee:

Numără frecvența fiecărei valori și reconstruiește vectorul în ordine.

Complexitate:

- $O(n + k)$ , unde  $k$  este intervalul de valori;
- memorie:  $O(k)$ ;
- stabil: poate fi stabil dacă este implementat cu poziții cumulative.

Întrebare probabilă:

"De ce Counting Sort nu este mereu alegerea cea mai bună?"

Răspuns:

"Pentru că depinde de intervalul valorilor. Dacă valorile sunt foarte mari sau negative fără normalizare, vectorul de frecvență devine ineficient sau greu de folosit."

### 3.8 Comparăție rapidă

Bubble Sort:

Simple, stabil,  $O(n^2)$ , bun pentru explicații.

Selection Sort:

Puține swap-uri,  $O(n^2)$ , instabil în varianta standard.

Insertion Sort:

Bun pe date aproape sortate, stabil,  $O(n)$  în caz bun,  $O(n^2)$  în caz rău.

Quick Sort:

Foarte rapid în practică,  $O(n \log n)$  mediu,  $O(n^2)$  rău, instabil.

Merge Sort:

Garantat  $O(n \log n)$ , stabil, dar cere memorie  $O(n)$ .

Counting Sort:

Foarte rapid pentru valori întregi într-un interval mic,  $O(n+k)$ , dar nu este algoritm prin comparații.

#### 4. ALGORITMI FUNDAMENTALI ȘI TEHNICI ALGORITMICE

Proiectul include și:

- algoritmi fundamentali;
- recursivitate;
- divide et impera;
- backtracking;
- greedy.

Răspuns bun dacă ești întrebat de ce le-ai adăugat:

"Am vrut ca platforma să nu rămână doar la sortări, ci să ofere o bază mai apropiată de ce învață elevii în problemele de tip PBIInfo."

Algoritmi fundamentali:

- parcurgere liniară;
- cifrele unui număr;
- divizori;
- CMMDC și CMMMC;
- numere prime și factorizare;
- Fibonacci;
- baze de numerație;
- căutare liniară și binară;
- vectori de frecvență;
- ciurul lui Eratostene.

Recursivitate:

O funcție se apelează pe ea însăși pentru o problemă mai mică. Are nevoie de caz de bază, pas recursiv și progres spre oprire.

Divide et Impera:

Împarți problema în subprobleme mai mici, le rezolvi și combini răspunsurile. Exemple: căutare binară, Merge Sort, Quick Sort.

Backtracking:

Construiești soluția pas cu pas. Dacă o alegere nu mai poate duce la soluție, revii și încerci altă valoare. Cuvinte-cheie:  $x[k]$ ,  $\text{valid}(k)$ ,  $\text{solutie}(k)$ .

Greedy:

Alegi la fiecare pas varianta locală cea mai bună și nu revii asupra deciziei. Trebuie demonstrat că alegerea locală duce la optim global.

Întrebare probabilă:

"Care este diferența dintre Greedy și Backtracking?"

Răspuns:

"Backtracking explorează variante și poate reveni. Greedy alege o variantă locală și merge mai departe fără revenire."

## 5. SECURITATE: CE TREBUIE SĂ POȚI EXPLICA

Parole:

Parolele nu sunt salvate în clar. Se folosește `password_hash`, iar la autentificare `password_verify`.

SQL Injection:

Interogările importante folosesc prepared statements cu `bind_param`.

XSS:

Output-ul dinamic se afișează cu `htmlspecialchars`, iar aplicația are Content Security Policy.

CSRF:

Formularele și cererile AJAX folosesc token CSRF.

Rate limiting:

AI-ul și acțiunile sensibile sunt limitate pe utilizator sau IP.

Sesiuni:

Există timeout de sesiune după inactivitate.

Răspuns general bun:

"Am implementat apărare pe mai multe straturi: prepared statements pentru SQL Injection, `htmlspecialchars` și CSP pentru XSS, token CSRF pentru formulare, `password_hash` pentru parole, rate limiting pentru abuz și timeout pentru sesiuni."

## 6. AI: CE TREBUIE SĂ APERI ÎN FAȚA COMISIEI

De ce Groq și Llama 3.3?

"Groq oferă latență foarte mică și API compatibil cu stilul OpenAI Chat Completions. Llama 3.3 70B este suficient de puternic pentru explicații didactice și generare de întrebări. Pentru un proiect academic, combinația este rapidă, accesibilă și ușor de migrat."

Ce se întâmplă dacă API-ul nu merge?

"Modulele AI sunt opționale. Dacă lipsește cheia sau API-ul răspunde cu eroare, restul platformei rămâne funcțional: teoria, vizualizatorul, grilele și profilul nu depind critic de AI."

Cum se evită întrebările proaste generate de AI?

"În prompt am introdus reguli stricte: să nu întrebe ce funcție din aplicație folosește un algoritm, să includă fragmente de cod când întreabă despre variabile și să pună întrebări rezolvabile din context. După generare, aplicația filtrează întrebările de memorare."

De ce trebuie să fii logat ca să dai testul AI?

"Pentru că testul AI salvează scorul și evoluția în timp. Dacă utilizatorul nu este autentificat, nu avem cui să asociem încercarea."

Ce înseamnă că AI-ul răspunde pe baza proiect\_documentatie?

"Aplicația caută fragmente relevante în indexul documentation\_index.json, construit din proiect\_documentatie. Aceste fragmente sunt introduse în prompt, iar modelul trebuie să răspundă prioritar pe baza lor."

## 7. PROFIL, PROGRES ȘI GAMIFICATION

Profilul utilizatorului afișează:

- lecții parcurse;
- progres mediu;
- grile rezolvate;

- acuratețe;
- streak;
- istoric AI quiz;
- badge-uri.

De ce contează:

Pentru învățare, feedback-ul imediat și progresul vizibil cresc motivația. Utilizatorul vede că a avansat, nu doar că a citit o pagină.

Întrebare probabilă:

"Gamification-ul nu este doar marketing?"

Răspuns:

"Nu, pentru că badge-urile sunt legate de acțiuni reale: grile rezolvate, lecții parcurse, algoritmi completați și streak. Nu recompensăm simpla navigare, ci progresul concret."

## 8. PWA ȘI DOCKER

PWA:

Aplicația are manifest.json și service worker. Asta permite instalarea ca aplicație și cache pentru asset-uri.

Răspuns scurt:

"PWA-ul ajută la acces rapid și la utilizarea unor resurse chiar dacă rețeaua este instabilă."

Docker:

Aplicația poate fi pornită din Docker, cu servicii pentru web, MySQL și phpMyAdmin.

Răspuns scurt:

"Docker reduce diferențele dintre calculatoare. Dacă mediul este definit în docker-compose, aplicația pornește mai previzibil decât prin configurări manuale."

## 9. ÎNTREBĂRI GRELE ȘI RĂSPUNSURI SCURTE

Întrebare: De ce ați ales PHP și nu Laravel, Node sau Django?

Răspuns: Pentru simplitate, compatibilitate cu WAMP/LAMP și focus pe conținut didactic. PHP 8 modern, folosit corect, este suficient pentru un proiect academic și permite deploy ușor.

Întrebare: De ce C++ în exemple?

Răspuns: Pentru că C++ este limbaj folosit frecvent în liceu, bacalaureat și algoritmică în România. Platforma web rulează în PHP, dar conținutul didactic este orientat spre C++.

Întrebare: Ce face proiectul diferit față de PBIInfo?

Răspuns: PBIInfo este foarte bun pentru probleme și teorie. OffByOne se concentrează pe vizualizare interactivă, feedback rapid, parcurhuri, profil și AI contextualizat.

Întrebare: Ce face proiectul diferit față de VisuAlgo?

Răspuns: VisuAlgo este foarte bun pe vizualizări, dar OffByOne adaugă limba română, AI, quiz, feedback pe cod, profil și integrare cu documentația proprie.

Întrebare: Ce risc are AI-ul?

Răspuns: Poate greși sau halucina. De aceea îl legăm de documentație, îl limităm prin prompt, validăm output-ul și îl tratăm ca asistent, nu ca autoritate absolută.

Întrebare: Cum demonstrați că aplicația ajută învățarea?

Răspuns: Deocamdată prin funcționalități și feedback preliminar. Pentru validare riguroasă, următorul pas ar fi un studiu A/B între metoda clasică și metoda cu vizualizare, quiz și AI.

Întrebare: De ce este important să vedem comparații și swap-uri?

Răspuns: Pentru că elevul înțelege costul algoritmului nu doar din formula  $O(n^2)$ , ci din numărul concret de operații făcute pe un exemplu.



Întrebare: Dacă aveți AI, mai are rost profesorul?

Răspuns: Da. AI-ul oferă feedback rapid și explicații, dar profesorul validează, ghidează și corectează înțelegerea de profunzime.

Întrebare: Ce înseamnă "OffByOne"?

Răspuns: Este un inside joke de programatori: eroarea off-by-one apare când greșești limita cu o poziție. Pentru o platformă de algoritmi, numele este memorabil și potrivit.

## 10. SIMULARE DE RĂSPUNSURI PE SORTĂRI

Întrebare: Care algoritm este cel mai bun?

Răspuns: Nu există un singur algoritm cel mai bun. Depinde de date: Merge Sort oferă  $O(n \log n)$  garantat și stabilitate, Quick Sort este rapid în practică, Insertion Sort merge bine pe date mici sau aproape sortate, Counting Sort e excelent când intervalul valorilor este mic.

Întrebare: De ce Quick Sort este rapid în practică dacă are caz rău  $O(n^2)$ ?

Răspuns: Pentru că în majoritatea cazurilor pivotul împarte destul de echilibrat datele, iar operațiile sunt locale și eficiente în memorie. Cazul rău se poate reduce prin alegerea mai bună a pivotului.

Întrebare: Care este diferența dintre Merge Sort și Quick Sort?

Răspuns: Ambele folosesc ideea divide et impera. Merge Sort împarte în jumătăți și interclasează, având  $O(n \log n)$  garantat, dar memorie  $O(n)$ . Quick Sort partiționează după pivot, e rapid în practică, dar poate ajunge la  $O(n^2)$ .

Întrebare: Ce înseamnă stabilitatea sortării?

Răspuns: Dacă două elemente au chei egale, o sortare stabilă le păstrează ordinea relativă inițială.

Întrebare: De ce Counting Sort nu este sortare prin comparații?

Răspuns: Pentru că nu compară perechi de elemente ca să decidă ordinea. Numără aparițiile valorilor și reconstruiește rezultatul.

Întrebare: Cum explici  $O(n \log n)$  intuitiv?

Răspuns: Apar  $\log n$  niveluri de împărțire, iar pe fiecare nivel se procesează în total aproximativ  $n$  elemente. De aceea apare produsul  $n \log n$ .

## 11. DEMO RECOMANDAT ÎN ZIUA PREZENTĂRII

### Demo 1: Laborator vizual

1. Deschide un algoritm, de exemplu Quick Sort.
2. Alege un număr mediu de elemente.
3. Pornește rularea.
4. Arată barele, pseudocodul, comparațiile și schimbările.
5. Explică: "Aici se vede concret costul algoritmului."

### Demo 2: Comparații

1. Alege un scenariu: date mari, date aproape sortate sau valori repetate.
2. Arată recomandarea.
3. Explică de ce nu există algoritm universal.

### Demo 3: Profesor AI

1. Întreabă: "De ce Quick Sort poate ajunge la  $O(n^2)$ ?"
2. Arată că răspunde în română.
3. Menționează că răspunsul folosește proiect\_documentatie.

### Demo 4: Test AI

1. Intră logat.
2. Pornește testul AI.
3. Arată că scorul este salvat în profil.

## Demo 5: Profil

1. Arată progresul, streak-ul și istoricul AI quiz.
2. Explică partea de învățare în timp.

## 12. CUM SĂ ÎNVEȚI CU NOTEBOOKLM

### Ziua 1:

Ascultă podcastul complet generat din acest ghid. Nu încerca să reții tot. Scopul este să înțelegi povestea proiectului.

### Ziua 2:

Cere NotebookLM:

"Simulează o comisie tehnică și pune-mi 15 întrebări despre securitate, AI și arhitectură."

### Ziua 3:

Cere NotebookLM:

"Întreabă-mă doar despre metode de sortare. Vreau întrebări scurte, iar după răspunsul meu corectează-mă."

### Ziua 4:

Cere NotebookLM:

"Fă-mi un podcast de 10 minute doar despre diferențiatorii proiectului față de PBIInfo, VisuAlgo, GeeksForGeeks și W3Schools."

### În dimineața prezentării:

Cere NotebookLM:

"Dă-mi o recapitulare audio de 7 minute cu cele mai importante 10 idei și 10 răspunsuri scurte pentru comisie."

### 13. REGULA DE AUR PENTRU RĂSPUNSURI

Nu răspunde cu detalii lungi dacă nu ți se cer. Răspunde în trei pași:

1. Ideea principală.
2. Exemplu din proiect.
3. De ce contează pentru utilizator.

Exemplu:

Întrebare: De ce aveți Profesor AI?

Răspuns bun:

"Pentru feedback rapid. În proiect, Profesorul AI folosește documentația locală din proiect\_documentatie și răspunde în română. Asta ajută elevul să clarifice imediat o confuzie, fără să părăsească platforma."

### 14. CE SĂ NU SPUI

Evită:

- "AI-ul răspunde mereu corect."
- "Platforma este mai bună decât toate celelalte."
- "Securitatea este perfectă."
- "Am ales PHP pentru că era cel mai ușor și atât."
- "Nu știu, dar cred că merge."

Înlocuiește cu:

- "AI-ul este ghidat de documentație și validat prin reguli, dar rămâne asistent."
- "Platforma se diferențiază prin integrare, limba română și AI contextualizat."
- "Am implementat măsuri importante: CSRF, prepared statements, CSP, hash parole, rate limiting."
- "Am ales PHP pentru compatibilitate, simplitate de deploy și potrivire cu un proiect academic."
- "Nu am măsurat încă formal, dar este o direcție clară de validare viitoare."

## 15. MESAJUL FINAL

"OffByOne Academy nu este doar un site despre sortări. Este un mediu de învățare în care elevul vede algoritmul, îl testează, primește feedback, își urmărește progresul și poate cere explicații în română. Asta transformă învățarea algoritmilor din memorare în înțelegere."